10월 4주차 sync

지난주에 한 것

- DiffKV가 느려지는 원인 조사
- RTX 3090에서 실행할 대안으로 TurboAttention을 제안받았으나, 공개된 공식 구현이 없어 후속 실험은 진행하지 않음

이번주에 한 것

- DiffKV 최적화: DiffKV-v1
  - attention과 engine의 불필요한 device 동기화 제거
  - 32개 layer가 각각 GPU에서 읽던 KV cache 길이를 한 번에 CPU로 복사하도록 변경
- DiffKV 프로파일링 과정 및 결과 정리
  - 실험 간 latency 편차와 FP16, DiffKV-v1 사이의 비정상적인 GEMM 시간 차이 조사
  - thermal throttling을 확인하고, base clock 고정과 온도 관리 후 재측정
  - FP16과 DiffKV가 공통 실행 가능한 14개 workload의 FP16, DiffKV, DiffKV-v1을 nsys로 kernel latency breakdown

목차

0. 실험 조건
  0.1. 측정한 workload
  0.2. Offline, fixed synthetic workload를 사용한 이유
1. DiffKV 최적화 (DiffKV-v1)
  1.1. 명시적인 동기화 제거
  1.2. Layer별 KV Cache length 계산 오버헤드 줄이기
  1.3. 결과
2. DiffKV 프로파일링 및 병목 식별
  2.1. 초기 측정에서 발견한 문제
  2.2. 원인 확인
  2.3. 해결 방법과 재측정
  2.4. Profiling results
  2.5. 결과 해석
3. 다음 주 계획

0. 실험 조건

- **GPU:** GPU0, NVIDIA GeForce RTX 3090, SM clock 1395MHz 고정
- **Dataset**: Fixed ISL/OSL synthetic data
- **모델:** Meta-Llama-3-8B, weight dtype float16, eager execution
- **비교 경로:** FP16 vLLM, DiffKV, DiffKV-v1
- **Workload:** B ∈ {8, 16, 32}, ISL ∈ {512, 1024, 2048}, OSL ∈ {512, 1024, 2048}의 27개 조합 중 FP16 및 DiffKV를 모두 실행 가능한 14개 case
  - B: Batch size (request 개수)
  - ISL: Input Sequence Length
  - OSL: Output Sequence Length

0.1. 측정한 workload

- 참고: DiffKV는 모든 case에서 돌아갔음

B	ISL ＼ OSL	512	1024	2048
8	512	측정	측정	측정
	1024	측정	측정	측정
	2048	측정	측정	측정
16	512	측정	측정	FP16 OOM
	1024	측정	측정	FP16 OOM
	2048	FP16 OOM	FP16 OOM	FP16 OOM
32	512	측정	FP16 OOM	FP16 OOM
	1024	FP16 OOM	FP16 OOM	FP16 OOM
	2048	FP16 OOM	FP16 OOM	FP16 OOM

0.2. Offline, fixed synthetic workload를 사용한 이유

기존 GSM8K 실행에서는 Llama-3-8B가 같은 문장이나 문제를 반복 출력하는 경우가 많았고, 따라서 같은 질문에서도 FP16과 DiffKV의 OSL이 서로 달라 공평한 성능 측정이 어려웠습니다. (e.g., 한 request에서 FP16은 반복 출력으로 1024토큰까지 생성했지만 DiffKV는 정상출력하여 292토큰에서 종료함)

이를 위해 offline으로 전체 B개 request를 한꺼번에 engine에 등록하고, 고정 synthetic data 입력을 사용했습니다. `temperature=0`, `ignore_eos=True`, `max_tokens=OSL` 로 매 요청마다 정확히 OSL개 토큰을 생성하도록 하였고, 매 step마다 고정된 B를 쭉 유지하고 preemption이 발생하지 않도록 하였습니다.

- Step 1: B개 request prefill
- Step 2: B개 request 1번째 decode
- Step 3: B개 request 2번째 decode
- …
- Step OSL: B개 request (OSL-1)번째 decode

1. DiffKV 최적화 (DiffKV-v1)

1.1. 명시적인 동기화 제거

https://github.com/130bb56/DiffKV/blob/main/vllm/model_executor/layers/sparse_attention_big_kernel.py#L362

▼ 기존 decode 경로 코드

```python
cache_ops.compress_and_append_cache_decode_phase(
    key,
    value,
    input_positions,
    kv_cache,
    slot_ids,
    block_tables,
    kv_len_tables,
    compress_config_tables,
    max_context_len,
    self.kv_buffer_size,
    self.layer,
    input_metadata.num_bits_k_high,
    input_metadata.num_bits_v_high,
    input_metadata.num_bits_k_low,
    input_metadata.num_bits_v_low,
    input_metadata.num_chunks_k_high,
    input_metadata.num_chunks_v_high,
    input_metadata.num_chunks_k_low,
    input_metadata.num_chunks_v_low,
    input_metadata.key_vec_size,
    input_metadata.val_vec_size,
    input_metadata.num_tokens_per_block_high,
    input_metadata.num_tokens_per_block_low)

torch.cuda.synchronize()
t1 = time.time()

# print('[DEBUG] SparsePagedAttention decode KV cache saved')

# key_cache_reshaped = torch.transpose(key_cache, 1, 2).contiguous().view(-1, block_size, self.head_size)
# key_cache_np = key_cache_reshaped.cpu().numpy()
# import numpy as np
# np.save("big_kernel_key_cache_layer_" + str(self.layer) + ".npy", key_cache_np)

# a new token might be appended in this forward pass
max_context_len += 1
# tmp_scores is a buffer to store the attention scores for each kv head
# TODO: we should optimize this
tmp_scores = torch.empty((batch_size, self.num_heads, max_context_len),
                         dtype=torch.float32, device=query.device)

if max_context_len <= _PARTITION_SIZE:
    num_partitions = 1
else:
    num_partitions = ((max_context_len + _PARTITION_SIZE - 1) // _PARTITION_SIZE) + 1
tmp_output = torch.empty(
    size=(batch_size, self.num_heads, num_partitions, self.head_size),
    dtype=output.dtype,
    device=output.device,
)
exp_sums = torch.empty(
    size=(batch_size, self.num_heads, num_partitions),
    dtype=torch.float32,
    device=output.device,
)
max_logits = torch.empty_like(exp_sums)

# TODO(@yanqi): Pass the following parameters from the framework. Now they are hardcoded.
attention_op = (ops.sparse_paged_attention_gqa if self.use_gqa_reuse
                else ops.sparse_paged_attention)
attention_op(
    slot_ids,
    input_positions,
    output,
    exp_sums,
    max_logits,
    tmp_output,
    tmp_scores,
    query,
    kv_cache,
    self.layer,
    self.num_kv_heads,
    self.scale,
    block_tables,
    kv_len_tables,
    max_context_len,
    input_metadata.num_bits_k_high,
    input_metadata.num_bits_v_high,
    input_metadata.num_bits_k_low,
    input_metadata.num_bits_v_low,
    input_metadata.num_chunks_k_high,
    input_metadata.num_chunks_v_high,
    input_metadata.num_chunks_k_low,
    input_metadata.num_chunks_v_low,
    input_metadata.key_vec_size,
    input_metadata.val_vec_size,
    input_metadata.num_tokens_per_block_high,
    input_metadata.num_tokens_per_block_low,
    self.alibi_slopes,
)

torch.cuda.synchronize()
t2 = time.time()
```

기존 decode path를 잘 보면 KV Cache를 압축하고 update하는 `compress_and_append_cache_decode_phase` 커널과 Attention 커널 사이의 모든 커널이 동일한 CUDA stream을 사용하고 있으므로, 명시적인 device sync (`torch.cuda.synchronize()`)를 제거해도 무방합니다.

1.2. Layer별 KV Cache length 계산 오버헤드 줄이기

DiffKV는 layer별로 Mixed KV Cache Quantization과 Pruning을 layer별로 non-uniform하게 적용하고 있으며, 그러면 layer별로 Attention에 들어갈 sequence length는 달라지게 됩니다. 이때 pruning 적용 후 GPU DRAM에 각 KV Cache의 길이를 저장하는 자료구조가 kv_len_tables입니다.

- kv_len_tables: per-request / layer / head max(KV Cache length)를 저장하는 자료구조

그러면 왜 layer별로 KV Cache length의 최대값을 알아야하냐가 궁금할 수 있습니다. 현재 Attention 커널은 긴 KV Cache를 Split-K방식으로 여러 block에 분산하여 병렬성을 높이는데, 이 과정에서 block단위로 max_logit과 exp_sum, attention score 등을 저장하고 이를 전체 Attention 결과로 aggregation 해야하므로 "각 layer에서 처리할 최대 KV Cache length"를 알아야 합니다. 이전 구현(AS-IS)와 최적화한 구현(TO-BE)을 통해 문제와 이를 해결하기 위해 적용한 최적화를 살펴보겠습니다.

1.2.1. AS-IS

먼저 modelrunner에서 layer별 최대 KV Cache length를 구하고 해당 length를 InputMetadata로 전달한 뒤, 각 layer의 Attention이 이를 사용합니다. 이때 이 매 decode layer마다 불리는 Attention에서 이 GPU VRAM에 있는 Maximum KV Cache Length `input_metadata.max_context_len` 를 CPU 변수 `max_context_len` 로 가져오는 과정에서 오버헤드가 발생하게 됩니다.

```python
# ModelRunner._prepare_decode()
max_context_len = torch.amax(
    # GPU Tensor, shape=[num_layers]
    torch.sum(kv_len_tables[slot_ids], dim=3), dim=(0, 2))

# SparsePagedAttention.forward()
max_context_len = input_metadata.max_context_len[self.layer].item()
```

따라서, 이를 매 Decode step 초기에 딱 한번만 전체 layer에 대한 Maximum KV Cache Length를 담는 CPU 리스트를 만들어주면 됩니다.

1.2.2. TO-BE

```python
# ModelRunner._prepare_decode()
max_context_len = torch.amax(
    torch.sum(kv_len_tables[slot_ids], dim=3),
    dim=(0, 2)
).cpu().tolist()

# SparsePagedAttention.forward()
max_context_len = input_metadata.max_context_len[self.layer]
```

이러면 매 layer마다 decode attention forward에서 GPU->CPU로 전송하던 오버헤드를 없애고, 한번에 bulk로 CPU의 list에 저장하여 사용할 수 있습니다.

https://github.com/130bb56/DiffKV/blob/bdefda19fadeef837549289a740c7f741dd07ae8/benchmarks/diffkv_v1.py#L59

구현은 `benchmarks/diffkv_v1.py` 에서 확인할 수 있습니다.

1.3. 결과

이 두 최적화를 적용한 결과 동기화 오버헤드가 크게 줄어들어 B=8, ISL=2048, OSL=2048 기준으로 기존 DiffKV 대비 E2E latency가 105.810s 에서 77.055s 로 27.18% 감소하였습니다.

B	ISL	OSL	Config	E2E latency	GEMM	Attention	KV Update	Other GPU kernels	Others
8	2048	2048	DiffKV	105.810	48.034 (45.40%)	18.442 (17.43%)	2.788 (2.64%)	2.253 (2.13%)	34.133 (32.26%)
			DiffKV-v1	**77.055**	48.028 (62.33%)	18.435 (23.92%)	2.789 (3.62%)	2.267 (2.94%)	**5.446 (7.07%)**

2. DiffKV 프로파일링 및 병목 식별

2.1. 초기 측정에서 발견한 문제

초기에는 같은 workload에서도 측정 간 latency가 크게 달라졌고, 비슷해야 할 FP16과 DiffKV-v1의 GEMM 실행시간이 큰 차이가 있었습니다. 따라서 측정 환경에서 어떤 문제가 있었는지 조사하였습니다.

아래는 (8, 2048, 2048)를 3회 측정한 결과입니다.

Config	E2E latency (s)	GEMM (s)
FP16	81.477 ± 3.703	54.123 ± 2.692
DiffKV-v1	73.775 ± 3.442	46.148 ± 2.011

2.2. 원인 확인

동일한 FP16 실행 안에서 **작업량이 같은 decode GEMM이 느려지는 시점**과 GPU clock, thermal flag를 대조했습니다. 대상은 GPU0의 B=8, ISL=2048, OSL=2048 첫 번째 측정, `measured_generation_r1` 입니다.

1. NSYS를 SQLite로 내보낸 `trace.sqlite` 의 `NVTX_EVENTS` 에서 decode `engine_step` 2047개를 추출했습니다. OSL 2048 중 첫 토큰은 prefill에서 생성하므로 decode는 2047 step이며, step 번호는 1부터 셌습니다.
2. `CUPTI_ACTIVITY_KIND_KERNEL.correlationId` 를 `CUPTI_ACTIVITY_KIND_RUNTIME` 의 CUDA launch와 연결하고, launch가 속한 decode NVTX 구간에 GEMM 커널을 배정했습니다. 각 커널의 `end - start` 를 합산해 step별 GEMM 시간을 계산했습니다.
3. 모든 decode step에서 B=8을 유지하므로 projection과 MLP GEMM의 M은 8, K와 N은 모델 차원으로 고정됩니다. 예를 들어 MLP gate/up은 (M, K, N)=(8, 4096, 28672)입니다. 누적 context 길이에 따라 작업량이 늘어나는 Attention과 달리, 이 GEMM의 크기는 decode 위치에 따라 커지지 않습니다. 실제 trace에서도 step마다 split-K reduce를 포함한 GEMM 호출 161개의 이름과 실행 구성이 같았습니다.
4. 약 200ms 간격의 `gpu.csv` 에서 KST timestamp를 Unix ns로 변환하고, SQLite의 `TARGET_INFO_SESSION_START_TIME.utcEpochNs` 를 빼 NSYS와 같은 상대 시간축으로 맞췄습니다. 같은 decode 구간에 속한 샘플에서 평균 SM clock과 `SW Thermal Slowdown` 활성 샘플 비율을 집계했습니다.

2047개 decode step을 다음 네 구간으로 나눠 통계를 집계했습니다.

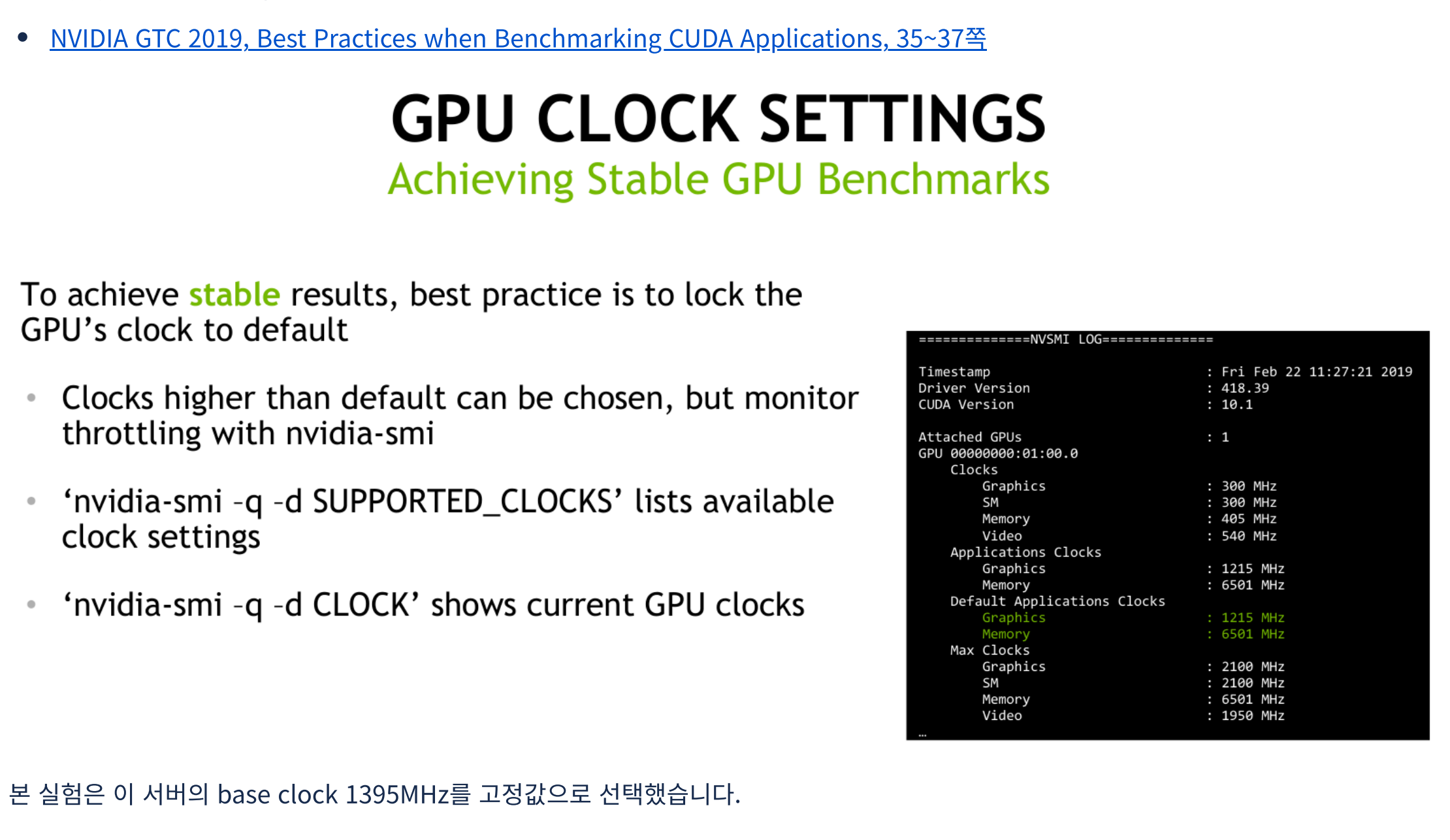
Decode step	GEMM (ms/step)	평균 GPU clock (MHz)	SW Thermal Slowdown
[1, 512]	19.344	1777.817	0/71 (0.00%)
[513, 1024]	24.819	1188.387	86/93 (92.47%)
[1025, 1536]	24.747	1169.235	98/98 (100.00%)
[1537, 2047]	24.610	1157.970	101/101 (100.00%)

동일한 크기와 구성의 GEMM이 느려진 구간에서 clock 하락과 드라이버의 thermal 제한 활성화를 함께 확인했으므로, 초기 비교에 thermal throttling이 발생했다고 판단했습니다.

2.3. 해결 방법과 재측정

측정 간 GPU clock 변동을 줄이기 위해 **GPU0에서 직접 조회한 base clock인 1395MHz로 clock을 고정**하고 재측정했습니다. NVIDIA도 안정적인 벤치마킹을 위해 GPU clock을 고정하도록 권고합니다.

- NVIDIA GTC 2019, Best Practices when Benchmarking CUDA Applications, 35~37쪽



GPU CLOCK SETTINGS

Achieving Stable GPU Benchmarks

To achieve stable results, best practice is to lock the GPU's clock to default

- Clocks higher than default can be chosen, but monitor throttling with nvidia-smi
- 'nvidia-smi -q -d SUPPORTED_CLOCKS' lists available clock settings
- 'nvidia-smi -q -d CLOCK' shows current GPU clocks

```
==============NVSMI LOG==============

Timestamp                           : Fri Feb 22 11:27:21 2019
Driver Version                      : 418.39
CUDA Version                        : 10.1

Attached GPUs                       : 1
GPU 00000000:01:00.0
    Clocks
        Graphics                    : 300 MHz
        SM                          : 300 MHz
        Memory                      : 405 MHz
        Video                       : 540 MHz
    Applications Clocks
        Graphics                    : 1215 MHz
        Memory                      : 6501 MHz
    Default Applications Clocks
        Graphics                    : 1215 MHz
        Memory                      : 6501 MHz
    Max Clocks
        Graphics                    : 2100 MHz
        SM                          : 2100 MHz
        Memory                      : 6501 MHz
        Video                       : 1950 MHz
…
```

본 실험은 이 서버의 base clock 1395MHz를 고정값으로 선택했습니다.

2.3.1. 재측정 과정

1. `nvidia-smi base-clocks` 으로 GPU의 base clock이 1395MHz임을 확인하고 SM clock을 고정했습니다.
   1.
   ```
   (dev) 130bb56@CASSLAB-Server15:~/DiffKV$ nvidia-smi base-clocks
   GPU 0:
           Graphics:       1395MHz
   GPU 1:
           Graphics:       1395MHz

   (dev) 130bb56@CASSLAB-Server15:~/DiffKV$ sudo nvidia-smi -i 0 --lock-gpu-clocks=1395,1395
   ```
2. 측정 전 온도를 확인하고, 온도가 높으면 충분히 대기합니다.
3. 실행 중 `nvidia-smi` 로 클럭, 온도, 전력과 thermal flag 등을 200ms 간격으로 기록합니다.
4. nsys의 decode 구간과 `nvidia-smi` 로그 timestamp를 맞춰, 실제 측정 구간에서 thermal throttling이 발생하면 해당 process를 kill하고, 충분히 GPU를 식혀준 후 다시 돌려주었습니다.

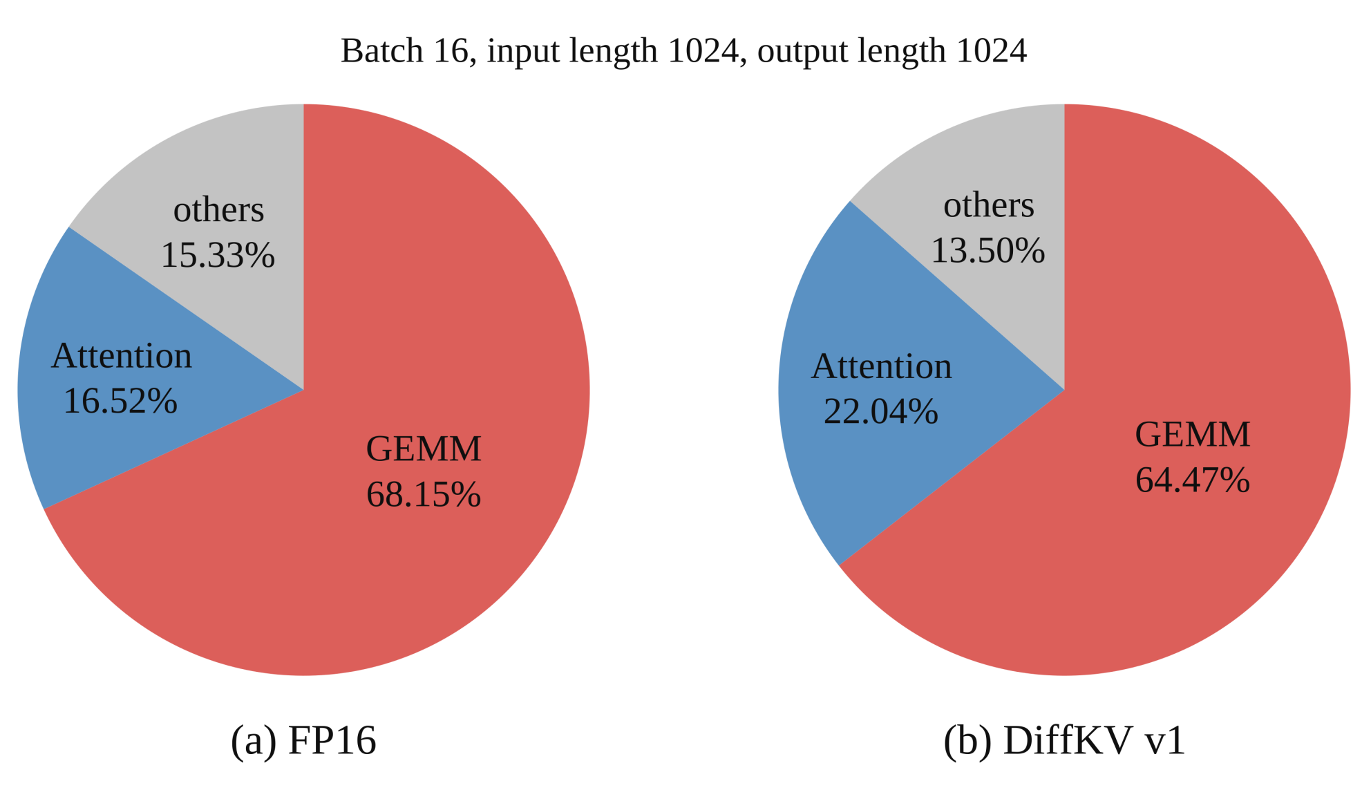
2.3.2. 결과

측정 조건	Config	E2E latency (s)	GEMM (s)
AS-IS	FP16	81.477 ± 3.703	54.123 ± 2.692
	DiffKV-v1	73.775 ± 3.442	46.148 ± 2.011
TO-BE	FP16	73.56781 ± 0.04043	47.91949 ± 0.00051
	DiffKV-v1	77.05523 ± 0.01397	48.02847 ± 0.00023

2.4. Profiling results

B	ISL	OSL	Config	E2E latency	GEMM	Attention	KV Update	Other GPU kernels	Others
8	512	512	FP16	**14.752**	11.993 (81.30%)	**0.813 (5.51%)**	**0.069 (0.47%)**	**0.520 (3.53%)**	**1.342 (9.09%)**
			DiffKV	20.978	12.011 (57.26%)	1.611 (7.68%)	0.537 (2.56%)	0.566 (2.70%)	6.216 (29.63%)
			DiffKV-v1	16.200	12.010 (74.13%)	1.610 (9.94%)	0.537 (3.32%)	0.567 (3.50%)	1.459 (9.01%)
8	512	1024	FP16	**29.372**	22.976 (78.22%)	**2.166 (7.37%)**	**0.134 (0.46%)**	**1.007 (3.43%)**	3.058 (10.41%)
			DiffKV	44.289	23.012 (51.96%)	3.925 (8.86%)	1.103 (2.49%)	1.091 (2.46%)	15.084 (34.06%)
			DiffKV-v1	32.071	23.008 (71.74%)	3.921 (12.23%)	1.104 (3.44%)	1.096 (3.42%)	**2.907 (9.07%)**
8	512	2048	FP16	**60.713**	44.942 (74.02%)	**6.475 (10.66%)**	**0.264 (0.43%)**	**1.982 (3.26%)**	6.977 (11.49%)
			DiffKV	85.704	45.021 (52.53%)	10.554 (12.31%)	2.347 (2.74%)	2.149 (2.51%)	25.482 (29.73%)
			DiffKV-v1	65.303	45.017 (68.94%)	10.543 (16.14%)	2.348 (3.60%)	2.158 (3.30%)	**5.163 (7.91%)**
8	1024	512	FP16	**16.688**	12.967 (77.70%)	**1.390 (8.33%)**	**0.073 (0.44%)**	**0.554 (3.32%)**	1.687 (10.11%)
			DiffKV	22.712	12.985 (57.17%)	2.321 (10.22%)	0.578 (2.54%)	0.600 (2.64%)	6.190 (27.25%)
			DiffKV-v1	17.850	12.983 (72.73%)	2.317 (12.98%)	0.578 (3.24%)	0.602 (3.37%)	**1.352 (7.57%)**
8	1024	1024	FP16	**32.035**	23.950 (74.76%)	**3.262 (10.18%)**	**0.138 (0.43%)**	**1.041 (3.25%)**	3.608 (11.26%)
			DiffKV	44.024	23.988 (54.49%)	5.285 (12.01%)	1.180 (2.68%)	1.129 (2.56%)	12.366 (28.09%)
			DiffKV-v1	34.190	23.986 (70.16%)	5.278 (15.44%)	1.181 (3.45%)	1.132 (3.31%)	**2.576 (7.54%)**
8	1024	2048	FP16	**64.899**	45.916 (70.75%)	**8.715 (13.43%)**	**0.268 (0.41%)**	**2.015 (3.11%)**	7.903 (12.18%)
			DiffKV	89.821	46.007 (51.22%)	13.184 (14.68%)	2.494 (2.78%)	2.186 (2.43%)	25.797 (28.72%)
			DiffKV-v1	69.537	46.002 (66.16%)	13.163 (18.93%)	2.495 (3.59%)	2.193 (3.15%)	**5.605 (8.06%)**
8	2048	512	FP16	**20.492**	14.972 (73.06%)	**2.616 (12.76%)**	**0.081 (0.40%)**	**0.621 (3.03%)**	2.180 (10.64%)
			DiffKV	26.857	14.997 (55.84%)	3.807 (14.17%)	0.675 (2.51%)	0.674 (2.51%)	6.667 (24.82%)
			DiffKV-v1	21.575	14.996 (69.51%)	3.804 (17.63%)	0.675 (3.13%)	0.675 (3.13%)	**1.404 (6.51%)**
8	2048	1024	FP16	**37.325**	25.955 (69.54%)	**5.605 (15.02%)**	**0.146 (0.39%)**	**1.108 (2.97%)**	4.465 (11.96%)
			DiffKV	51.166	26.007 (50.83%)	8.037 (15.71%)	1.343 (2.62%)	1.201 (2.35%)	14.501 (28.34%)
			DiffKV-v1	39.372	26.005 (66.05%)	8.032 (20.40%)	1.343 (3.41%)	1.206 (3.06%)	**2.743 (6.97%)**
8	2048	2048	FP16	**73.568**	47.919 (65.14%)	**13.216 (17.96%)**	**0.276 (0.38%)**	**2.083 (2.83%)**	9.975 (13.56%)
			DiffKV	105.810	48.034 (45.40%)	18.442 (17.43%)	2.788 (2.64%)	2.253 (2.13%)	34.133 (32.26%)
			DiffKV-v1	77.055	48.028 (62.33%)	18.435 (23.92%)	2.789 (3.62%)	2.267 (2.94%)	**5.446 (7.07%)**
16	512	512	FP16	**17.963**	13.727 (76.42%)	**1.656 (9.22%)**	**0.072 (0.40%)**	**0.577 (3.21%)**	1.913 (10.65%)
			DiffKV	24.256	13.750 (56.69%)	2.893 (11.93%)	0.551 (2.27%)	0.623 (2.57%)	6.401 (26.39%)
			DiffKV-v1	19.384	13.748 (70.93%)	2.891 (14.91%)	0.551 (2.84%)	0.625 (3.22%)	**1.551 (8.00%)**
16	512	1024	FP16	**35.330**	25.471 (72.09%)	**4.363 (12.35%)**	**0.136 (0.39%)**	**1.089 (3.08%)**	4.230 (11.97%)
			DiffKV	51.427	25.524 (49.63%)	7.084 (13.77%)	1.124 (2.19%)	1.171 (2.28%)	16.448 (31.98%)
			DiffKV-v1	38.023	25.522 (67.12%)	7.075 (18.61%)	1.124 (2.96%)	1.177 (3.10%)	**3.085 (8.11%)**
16	1024	512	FP16	**21.792**	15.731 (72.19%)	**2.795 (12.83%)**	**0.080 (0.37%)**	**0.645 (2.96%)**	2.517 (11.55%)
			DiffKV	28.426	15.762 (55.45%)	4.217 (14.83%)	0.598 (2.10%)	0.696 (2.45%)	7.115 (25.03%)
			DiffKV-v1	23.143	15.760 (68.10%)	4.214 (18.21%)	0.598 (2.58%)	0.697 (3.01%)	**1.852 (8.00%)**
16	1024	1024	FP16	**40.499**	27.476 (67.84%)	**6.592 (16.28%)**	**0.145 (0.36%)**	**1.156 (2.85%)**	5.080 (12.54%)
			DiffKV	53.823	27.539 (51.17%)	9.606 (17.85%)	1.208 (2.24%)	1.248 (2.32%)	14.142 (26.28%)
			DiffKV-v1	42.794	27.538 (64.35%)	9.601 (22.43%)	1.207 (2.82%)	1.251 (2.92%)	**3.153 (7.37%)**
32	512	512	FP16	**22.806**	15.593 (68.37%)	**3.184 (13.96%)**	**0.082 (0.36%)**	**0.701 (3.07%)**	3.204 (14.05%)
			DiffKV	29.982	15.609 (52.06%)	5.081 (16.95%)	0.561 (1.87%)	0.756 (2.52%)	7.925 (26.43%)
			DiffKV-v1	24.140	15.606 (64.65%)	5.077 (21.03%)	0.560 (2.32%)	0.756 (3.13%)	**2.100 (8.70%)**

2.5. 결과 해석



(a) FP16

(b) DiffKV v1

Attention의 비율이 DiffKV-v1이 FP16 대비 훨씬 높으므로, Attention을 최적화 할 필요가 있습니다.
아직 v1으로도 FP16 대비 속도 향상은 없었습니다.

3. 다음 주 계획

- DiffKV Attention ncu로 병목 진단 후 커널 최적화